NKT
Tag 2: Objektorientierung

Alexa Katharina-Ronja Brockmöller, Tobias Wrede

9. März 2026

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Scanner
2. Vererbung
   - Klasse Object
   - Überschreiben von Methoden
3. Interfaces
4. Heap und Stack
   - Stack
   - Heap

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Scanner
2. Vererbung
   - Klasse Object
   - Überschreiben von Methoden
3. Interfaces
4. Heap und Stack
   - Stack
   - Heap

Klassen

- Eine Klasse definiert einen Datentyp
- beinhaltet Attribute (Zustand)
- enthält Methoden (Verhalten)
- Mit new können wir Objekte von Klassen erstellen

```
Person p1 = new Person();
Person p2 = new Person("Superman");
```

Klassenattribute

```
class Person {
    private String name = "Kein Name";
    private int alter;
}
```

- Attribute beschreiben den Zustand eines Objektes
- Zugriff auf Attribute über Methoden (Getter und Setter)
- Schlüsselwort final: Attribut kann nur einmal zugewiesen werden

Methoden in Klassen

Heinrich Heine Universität Düsseldorf

- Eine Methode zur Konstruktion eines Objektes: Konstruktor
- Wird nach new aufgerufen
- Konstruktoren haben den selben Namen wie die Klasse
- Wenn kein Konstruktor definiert ist, wird ein ein leerer Konstruktor erstellt
- Innerhalb von Methoden kann auf das Objekt mit this zugegriffen werden
- Setter und Getter kontrollieren den Zugriff auf Attribute

Methoden in Klassen

```
class Person {
    private String name = "Kein Name";
    private int alter;

    Person(String name, int alter){
        this.name = name;
        this.alter = alter;
    }
    String getName(){
        return name;
    }
    void setName(String name){
        this.name = name;
    }
}
```

Sichbarkeiten

- public: Sichtbar für alle Klassen
- private: Nur in der Klasse selbst sichtbar
- protected: Nur die Klasse selbst und Unterklassen sichtbar
- package-private (nichts): Nur für Klassen im selben Package (Ordner) sichtbar

Methoden Überladen

Heinrich Heine Universität Düsseldorf

Überladen: Mehrere Methoden mit dem selben Namen, aber unterschiedlichen Parametern. Auch bei Konstruktoren möglich.

```
int sum(int a, int b) {
    return a + b;
    }

double sum(double a, double b) {
    return a + b;
    }
```

Nun können wir die Methode sum mit int wie auch mit double verwenden.

Klassenattribute und Klassenmethoden

Heinrich Heine Universität Düsseldorf

- Schlüsselwort static: Attribut oder Methode gehört zur Klasse, nicht zum Objekt
- Zugriff auf statische Attribute und Methoden über Klassenname
- Beispiel: Math.PI, Math.sqrt()

Records und Enums

Enums: Aufzählungsklasse, definiert eine feste Menge von Konstanten

```
enum Seasons {
    WINTER,
    SPRING,
    SUMMER,
    AUTUMN;
}
```

Records: Klasse, die nur Daten enthält

- besitzt Getter und Konstruktor, sowie final Attribute
- Konstruktor setzt standardmäßig alle Attribute, kann aber auch überschrieben werden

```
record Person(String name, int alter) {}
```

Scanner & Dateien

Wir können Scanner aber auch auf spezifische Dateien setzen. Dafür brauchen wir jedoch den Dateipfad.

```
Scanner scanner = new Scanner(dateipfad);
```

Achtung! Wenn an diesem Pfad keine Datei liegt, wird unser Programm abstürzen. Wir sollten deshalb einen *try-catch* Block benutzen:

```
try{
    Scanner scanner = new Scanner("/NKT/wichtigeDatei.txt");
} catch(Exception e){
    e.printStacktrace();
}
```

Standardeingabe

Wir können auch die Standardeingabe benutzen, um unser Programm mit Daten aufzurufen. Dafür benötigen wir Scanner:

```
import java.util.Scanner;
class Wuerfel {
    static void main(String[] args) {
        Scanner stdin = new Scanner(System.in);
        while(stdin.hasNext()){
            int number = Integer.parseInt(stdin.nextLine());
            while(number > 0){
                for(int i = 0; i < number; i++){
                    System.out.print("*");
                }
                number--;
                System.out.println();
```

Shallow und Deep Copy

Heinrich Heine Universität Düsseldorf

- Object.clone kopiert ein Objekt.
- Erstellt bei Arrays nur eine shallow copy, d.h. die Referenzen auf die Elemente werden kopiert, nicht die Elemente selbst.
- Klappt somit nur bei primitivan oder immutable Datentypen
- Eine Deep Copy erstellt eine neue Instanz eines Objektes, inklusive aller Objekte, auf die es referenziert.
- Deep Copy muss manuell implementiert werden

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Scanner
2. Vererbung
   - Klasse Object
   - Überschreiben von Methoden
3. Interfaces
4. Heap und Stack
   - Stack
   - Heap

Vererbung

- Eine Klasse kann von einer anderen Klasse erben, um deren nicht private Attribute und Methoden zu nutzen.
- Schlüsselwort: extends
- Jede Klasse kann nur von einer Klasse erben
- Zugriff auf Oberklasse mit Schlüsselwort super
- Im Konstruktor der Unterklasse muss der Konstruktor der Oberklasse mit super() aufgerufen werden

```
class Student extends Person {
    private String matrikelnummer;
}
```

Student erbt von Person, hat also die Attribute name und alter sowie die Methoden getName()

Klasse Object

Heinrich Heine Universität Düsseldorf

- Die Klasse Object ist die Basisklasse für alle Klassen in Java.
- Alle Klassen erben standardmäßig von Object.
- Object hat einige wichtige Methoden, die von allen Klassen geerbt werden:
  - toString()
  - equals()
  - hashCode()

Überschreiben von toString()

- Eine geerbte Methode, kann in der Unterklassen eine andere Funktionalität erhalten. Dafür wird diese überschrieben.
- In unserem Person Beispiel haben wir die toString() Methode, die wir von Object geerbt haben, überschrieben.
- Wenn wir nun auf den Person Objekten die toString() Methode aufrufen wird nun unsere neue Methode genutzt, da die Alte überschrieben ist.

```
@Override
public String toString(){
    return name;
}
```

[1]

[1] @Override Annotation nicht nötig, dennoch sinnvoll

equals-Methode

==:

- Vergleicht Werte von primitiven Datentypen
- Vergleicht bei Objekten die Referenzen, also ob beide Variablen auf dasselbe Objekt zeigen

equals():

- Methode der Klasse Object, kann von Klassen überschrieben werden, um den Vergleich von Objekten zu ermöglichen
- Standardmäßig als == definiert, jedocch überschreibbar
- Beispiel: String Klasse überschreibt equals(), um den Vergleich von Strings zu ermöglichen

```
String a = "Hallo";
String b = "Hallo";
System.out.println(a.equals(b)); // True
System.out.println(a == b); // False
```

declared type vs. actual type

Heinrich Heine Universität Düsseldorf

- Einer Variablen kann nicht nur der Wert des deklarierten Typs zugewiesen werden, sondern auch der Wert eines Untertyps.
- Beispiel: Person p = new Student(); declared type: Person, actual type: Student
- p ist vom Typ Person, aber tatsächlich eine Instanz von Student.
- Methodenaufrufe auf p nutzen die Methoden von Student, wenn sie überschrieben wurden, da die tatsächliche Instanz von Student ist.
- Es lassen sich jedoch nur Methoden aufrufen, die im deklarierten Typ (Person) definiert sind

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Scanner
2. Vererbung
   - Klasse Object
   - Überschreiben von Methoden
3. Interfaces
4. Heap und Stack
   - Stack
   - Heap

Interfaces

Was sind Interfaces?

- Von Interfaces können keine Objekte erstellt werden
- Sie beschreiben die Schnittstelle einer Klasse, d.h. welche Methoden sie besitzen muss
- können von Klassen implementiert werden
- schreiben Klassen vor, welche Methoden sie besitzen müssen
- eine Klasse kann beliebig viele Interfaces implementieren
- Methoden in Interfaces sind standardmäßig public und abstract, d.h. sie müssen von der Klasse implementiert werden

Beispiel für ein Interface:

```
interface Item {
    double weight();
}
```

[2]

[2] *public* ist in Methoden, die von einem Interface vorgeschrieben werden nicht nötig

Interface implementieren

Heinrich Heine Universität Düsseldorf

Wie implementiert man ein Interface?

```
1    public class Schwert implements Item{
2        //inhalt der Klasse
3    }
```

Nun muss die Klasse Schwert eine Methode *weight()* implementieren, die einen *double* zurück gibt.

Interface Klasse A

```
class Schwert implements Item{
    String name;
    static final double WEIGHT = 7;
    Schwert(String name){
        this.name = name;
    }
    double weight(){
        return WEIGHT;
    }
}
```

Interface Klasse B

```
class Heiltrank implements Item {
    String name;
    int count;
    static final double WEIGHT = 2.5;
    Heiltrank(String name, int count){
        this.name = name;
        this.count = count;
    }
    double weight(){
        return count * WEIGHT;
    }
}
```

Abstrakte Klassen

Heinrich Heine Universität Düsseldorf

- Abstrakte Klassen können nicht instanziiert werden, können aber Methoden implementieren.
- Sie können auch abstrakte Methoden enthalten, die von Unterklassen implementiert werden müssen.
- Unterklassen von abstrakten Klassen müssen alle abstrakten Methoden implementieren, es sei denn, sie sind selbst abstrakt.

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Scanner
2. Vererbung
   - Klasse Object
   - Überschreiben von Methoden
3. Interfaces
4. Heap und Stack
   - Stack
   - Heap

Heap und Stack

Heinrich Heine Universität Düsseldorf

Wo werden eigentlich die Daten gespeichert?

- Stack
- Heap

Stack

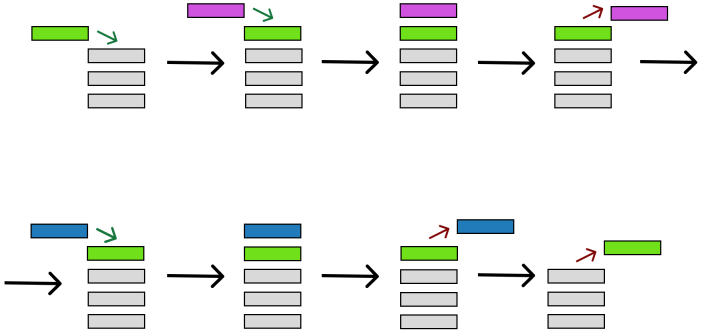
Heinrich Heine Universität Düsseldorf

Auf dem Stack Speichern wir primitive Datentypen und Referenzen. Auch Rücksprungadressen etc. (siehe Rechenarchitektur).

- float
- int
- double
- boolean
- char

Der Stack ist schneller erreichbar als der Heap. Dafür aber deutlich kleiner.

Stack - Zugriff



LIFO - Last In first Out

Heap

Heinrich Heine Universität Düsseldorf

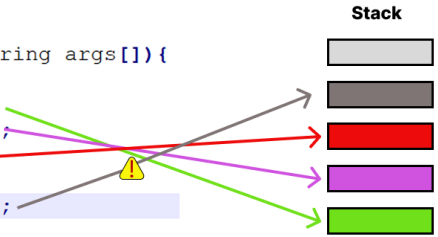
Auf dem Heap Speichern wir Objekte.

- Arrays
- String
- Integer
- Double
- Person
- eigene Klassen

Der Heap ist im Gegensatz zum Stack viel größer. Dafür aber langsamer.

Heap

```
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```



Vom Objekt wird nur eine Referenz gespeichert!

Referenzen

```
1    Person a = new Person();
2        Person b = new Person();
3        System.out.println(a == b);
```

Wird true oder false ausgegeben?

Referenzen

```
        Person a = new Person();
        Person b = new Person();
    System.out.println(a == b);
```

Wird true oder false ausgegeben?
→ Es wird false ausgegeben

```
        System.out.println(a);          //Person@7ad041f3
        System.out.println(b);          //Person@251a69d7
```

Heap 01

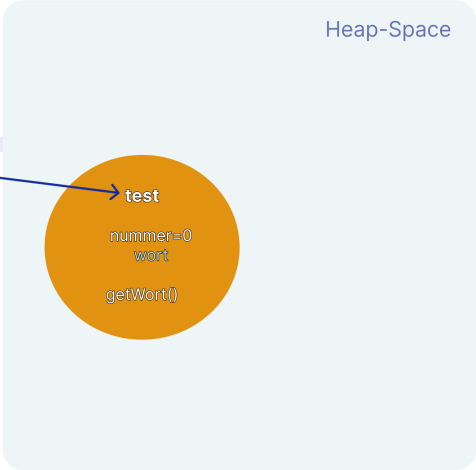
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();



    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```

Heap-Space

Heap 02

- Durch Aufruf des Konstruktors wird ein neues Objekt auf dem Heap erzeugt

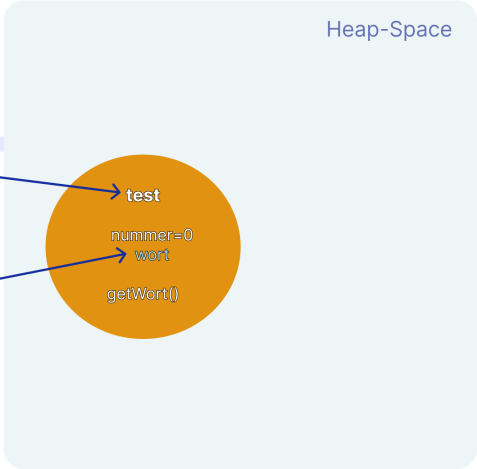
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Heap 03

- Gilt auch für Objekte in einem Objekt
- Jede Zuweisung gibt nur die Referenz auf das Objekt weiter!

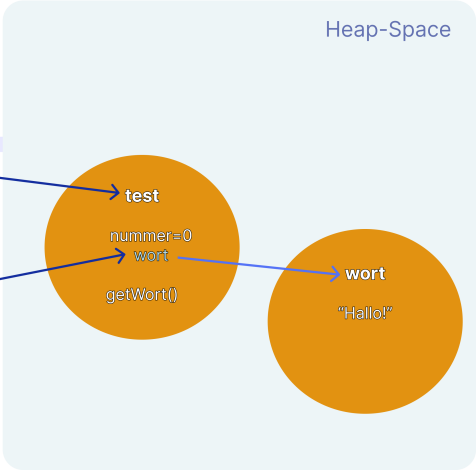
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Heap 04

- Shallow Copy:
  Wenn das Objekt kopiert wird, werden nur Referenzen auf andere Objekte kopiert

- Deep Copy:
  Die Objekte im Objekt sind echte Kopien

```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```

```java
public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```